Evaluación de Conocimientos y Diagnóstico

Módulo: Sistemas Operativos de Tiempo Real (RTOS) — Caso de Estudio: **FreeRTOS**
Diseñado para estudiantes de Ingeniería Electrónica y Computación Embebida

1. Diagnóstico de Conocimientos Previos (2 Preguntas Abiertas)

Pregunta D1: Arquitectura de Control Físico vs. Planificación Multitarea

En el desarrollo de firmware tradicional para microcontroladores sin sistema operativo, se suele utilizar una arquitectura de *Ejecutivo Cíclico* (bucle infinito main() combinado con rutinas de interrupción ISR).
Consigna: Explique cuál es la diferencia fundamental entre este enfoque secuencial y la planificación multitarea preventiva en un RTOS como FreeRTOS. Detalle por qué la latencia de respuesta ante eventos externos críticos se vuelve impredecible y difícil de escalar cuando aumenta la complejidad del programa en un Ejecutivo Cíclico.

Criterio de Evaluación y Diagnóstico Esperado:

Propósito de Evaluación: Identificar si el estudiante comprende las limitaciones de la programación secuencial por sondeo (polling), el concepto de latencia determinista de respuesta y la necesidad de un planificador preventivo guiado por prioridades para garantizar el cumplimiento estricto de *deadlines* en tiempo real.

Pregunta D2: Contexto de Ejecución de la CPU y la Memoria Pila (Stack)

En procesadores como los ARM Cortex-M, el flujo de ejecución depende del banco de registros internos de la CPU (R0-R12, SP, LR, PC, xPSR) y del puntero de pila (Stack Pointer).
Consigna: Justifique la importancia técnica de la memoria Pila (Stack) en un entorno multitarea. ¿Por qué es strictly necesario que cada tarea creada en FreeRTOS disponga de un bloque de memoria Pila privado e independiente en la RAM, y qué sucede con los registros del procesador durante un *cambio de contexto* (Context Switch)?

Criterio de Evaluación y Diagnóstico Esperado:

Propósito de Evaluación: Verificar los conocimientos previos en arquitectura de computadores, específicamente en apilamiento de registros, aislamiento de variables locales en cada hilo y preservación del estado del procesador (TCB / pxTopOfStack) para evitar la corrupción de datos entre tareas concurrentes.

2. Evaluación de Conceptos Básicos de FreeRTOS (5 Preguntas de Opción Múltiple)

Pregunta 1: Algoritmo de Planificación del Kernel

¿Cuál es la responsabilidad principal del planificador preventivo con tiempo compartido (*Prioritized Preemptive Scheduling with Time Slicing*) configurado por defecto en FreeRTOS?

A) Asignar memoria dinámica en la RAM cada vez que finaliza la ejecución de una tarea.

B) Garantizar que la tarea de mayor prioridad en estado *Ready* obtenga inmediatamente la CPU, alternando por turnos (Round-Robin) entre tareas de igual prioridad.

C) Ejecutar todas las tareas creadas en un orden estrictamente secuencial sin posibilidad de interrupción.

D) Deshabilitar permanentemente las interrupciones de hardware durante la ejecución de tareas de usuario.

Respuesta Correcta: B
Justificación: *En FreeRTOS, el planificador se rige por prioridades estrictas: siempre selecciona la tarea en estado Ready de mayor prioridad para pasar a estado Running. Si existen múltiples tareas listas con la misma prioridad máxima, el kernel distribuye el tiempo equitativamente entre ellas mediante Time Slicing en cada interrupción de reloj (Tick).*

Pregunta 2: Estructura de Control de Tareas (TCB)

¿Qué estructura interna utiliza FreeRTOS en la memoria RAM para gestionar el estado, prioridad y puntero de pila de cada tarea individual?

A) TCB (*Task Control Block*).

B) NVIC (*Nested Vectored Interrupt Controller*).

C) ISR (*Interrupt Service Routine*).

D) MPU (*Memory Protection Unit*).

Respuesta Correcta: A
Justificación: *El Task Control Block (TCB, definido internamente como tskTCB) almacena toda la información crítica de una tarea: el puntero al tope de la pila (pxTopOfStack, primer miembro obligatorio de la estructura), el estado de la tarea, su prioridad base y actual, y sus nodos de lista de sincronización.*

Pregunta 3: Mutexes y Solución a la Inversión de Prioridades

¿Cuál es la diferencia técnica fundamental entre un *Mutex* y un *Semáforo Binario* estándar en FreeRTOS cuando se utilizan para proteger recursos compartidos?

A) El Mutex consume menos memoria RAM que el semáforo binario.

B) El Mutex permite que múltiples tareas accedan simultáneamente al mismo periférico.

C) El Mutex implementa el protocolo de *Herencia de Prioridades* (Priority Inheritance) para evitar la inversión ilimitada de prioridades.

D) El Mutex es el único mecanismo que se puede 'entregar' (Give) desde una Rutina de Servicio de Interrupción (ISR).

Respuesta Correcta: C
Justificación: *Aunque ambos sirven para exclusión mutua, los Mutexes en FreeRTOS incluyen Herencia de Prioridades: si una tarea de alta prioridad intenta tomar un Mutex retenido por una tarea de baja prioridad, el kernel eleva temporalmente la prioridad de la tarea de baja prioridad al nivel de la tarea bloqueada, garantizando que libere el recurso lo antes posible sin interferencia de tareas de prioridad intermedia.*

Pregunta 4: Procesamiento Diferido de Interrupciones (Deferred Processing)

¿Cuál es la práctica de diseño recomendada en FreeRTOS para gestionar interrupciones de hardware complejas o de alta tasa de transferencia de datos?

A) Realizar todo el procesamiento de datos y algoritmos pesados directamente dentro de la ISR antes de borrar la bandera.

B) Mantener la ISR lo más corta posible, atendiendo solo el hardware urgente y desbloqueando una tarea diferida de alta prioridad mediante funciones con sufijo ...FromISR().

C) Suspender el planificador llamando a vTaskSuspendAll() dentro de la ISR.

D) Asignar memoria dinámica dinámicamente mediante pvPortMalloc() dentro de la interrupción.

Respuesta Correcta: B
Justificación: *La regla de oro en RTOS establece que las ISRs deben ser extremadamente breves para no introducir jitter ni bloquear otras interrupciones del sistema. La ISR realiza la acción mínima de hardware y delega (difiere) el procesamiento pesado a una tarea RTOS dedicada de alta prioridad mediante primitivas seguras para ISR (como xSemaphoreGiveFromISR o vTaskNotifyGiveFromISR).*

Pregunta 5: Tarea Inactiva (Idle Task) y Modo de Bajo Consumo

¿Qué función cumple la *Idle Task* creada automáticamente por FreeRTOS y cuál es la ventaja del modo *Tickless Idle*?

A) Recompilar automáticamente el firmware cuando detecta errores de sintaxis en la memoria Flash.

B) Asegurar que la CPU siempre tenga una tarea que ejecutar en prioridad 0 y reducir el consumo energético al suprimir las interrupciones periódicas del Tick durante periodos prolongados de inactividad.

C) Duplicar la frecuencia del oscilador de la CPU cuando el sistema está inactivo.

D) Forzar a que todas las tareas se ejecuten de forma cooperativa en lugar de preventiva.

Respuesta Correcta: B
Justificación: *Al iniciar el planificador (vTaskStartScheduler), FreeRTOS crea automáticamente la Idle Task con prioridad 0 para liberar memoria de tareas eliminadas y evitar que la CPU quede sin instrucciones. En sistemas alimentados por batería, el modo Tickless Idle detiene las interrupciones del temporizador SysTick mientras el sistema está inactivo, poniendo el microcontrolador en modos profundos de bajo consumo.*

3. Evaluación de Aplicaciones Industriales de FreeRTOS (3 Preguntas Abiertas)

Pregunta A1: Inteligencia Artificial en el Borde (Edge AI & TinyML)

En aplicaciones avanzadas de Edge AI sobre microcontroladores (como el ARM Cortex-M7 a 480 MHz en la plataforma STM32H7), se ejecutan modelos de Redes Neuronales Convolucionales (CNN) cuantizadas a INT8 mediante bibliotecas como *TensorFlow Lite for Microcontrollers* y *CMSIS-NN*.
Consigna: Explique cómo interactúa el Kernel de FreeRTOS con la tarea de inferencia de IA. En particular, analice la importancia de la **asignación estática de memoria pools** para el búfer tensor_arena y el papel del modo **Tickless Idle** para lograr determinismo en la latencia de inferencia (p. ej., 13.8 ms) y bajo consumo energético.

Puntos Clave y Criterios de Evaluación Esperados:

Guía de Respuesta / Criterios de Evaluación:
• *Asignación Estática:* Evita la fragmentación del heap y garantiza que los tensores del modelo dispongan de memoria determinista libre de fallos dinámicos (configSUPPORT_STATIC_ALLOCATION = 1).
• *Determinismo y Latencia:* FreeRTOS asigna prioridades elevadas a la tarea de inferencia y utiliza transferencias por DMA de cero copia para no desperdiciar ciclos de CPU.
• *Tickless Idle:* Suprime interrupciones de reloj innecesarias durante los intervalos de inactividad entre ráfagas de inferencia, maximizando la eficiencia energética (0.92 mJ por inferencia).

Pregunta A2: Control Digital de Motores y Electrónica de Potencia

Sistemas industriales como servodrives de robótica, inversores solares y tracción eléctrica emplean microcontroladores de tiempo real (como la familia C2000Ware de Texas Instruments) ejecutando FreeRTOS.

Consigna: Describa cómo conviven en el mismo microcontrolador los lazos de control de corriente ultrarrápidos (*Fast Current Loop* / FOC) y las pilas de comunicación industrial (CAN-FD / EtherCAT). Justifique además el valor de la certificación de seguridad funcional (p. ej., SAFERTOS bajo normas IEC 61508 SIL 3 e ISO 26262 ASIL D) en estos entornos críticos.

Puntos Clave y Criterios de Evaluación Esperados:

Guía de Respuesta / Criterios de Evaluación:

• *Gestión Multitarea:* Los lazos de control crítico (FOC/FCL) son atendidos mediante interrupciones de hardware prioritarias o tareas diferidas de la máxima prioridad RTOS, mientras que las comunicaciones (CAN/EtherCAT) se ejecutan a prioridades medias/bajas.

• *Seguridad Crítica (SAFERTOS):* Garantiza tolerancia a fallos, aislamiento en MPU y verificación formal requerida por estándares automotrices e industriales para prevenir catástrofes físicas ante errores de software.

Pregunta A3: Automatización e Impresoras Térmicas Industriales

En líneas de ensamblaje y logística de alta demanda, la impresión térmica continua suele enfrentar cuellos de botella como el desbordamiento de búferes de comunicación y la falta de sincronización entre el avance de papel y el calentamiento del cabezal.

Consigna: Analice de qué manera la implementación de un RTOS (como FreeRTOS o On-Time RTOS) resuelve estos problemas mediante la gestión de colas FIFO, semáforos de sincronización y temporizadores de precisión, reduciendo las paradas no programadas y mejorando el rendimiento global del sistema.

Puntos Clave y Criterios de Evaluación Esperados:

Guía de Respuesta / Criterios de Evaluación:

• *Estructuras FIFO y Colas:* Las colas de mensajes de FreeRTOS previenen la pérdida de datos de impresión procedentes de interfaces Ethernet/USB mediante sincronización segura y control de flujo.

• *Sincronización Determinista:* Semáforos contadores y timers coordinan el movimiento del motor paso a paso con los pulsos térmicos del cabezal, eliminando errores de impresión y paradas por desalineamiento.